

Booleans

CSCI 1101

2024-02-06

The Boolean Type

Previously, we met a type with exactly one element. (Which one?)

Inspecting a value of this type is pointless because we know what we will find before we even look. (What is it?)

So the smallest number of values that an “interesting” type can have is two.

This corresponds to learning the answer to a *yes-or-no* question.

Python has a two-value type called `bool`, whose elements are `True` and `False`.

It is named for *George Boole*, a mathematician who devised an algebra for logic.

Predicates

An expression or function that evaluates to a `bool` is called a **predicate**.

Python has many useful built-in predicates.

For example, the following functions from the `str` module all have signature `(str) --> bool`:

```
str.isdigit    # are all characters numeric?
str.isalpha    # are all characters alphabetic?
str.isupper    # are all characters uppercase?
str.islower    # are all characters lowercase?
```

Warning: they all answer `False` on the empty string.

To test whether one string occurs as a **substring** of another, we use the operator `_in_` : `(str , str) --> bool`.

```
"do" in "Bowdoin"      # occurs as a substring
"win" in "Bowdoin"     # characters must be consecutive
```

Equality Predicates

Given two elements *of the same type* we can test whether they are equal using the **equality predicate**, `_==_` (read “is equal to”).

```
"Bow" + "doin" == "Bowdoin"
```

```
6 * 7 == 30 + 12
```

```
False == True
```

```
"Hello" == "hello"
```

Warning: Equality testing between elements *of different types* may or may not do what you expect. Python sometimes implicitly applies type conversion functions like we met last week.

```
3 == 3.0           # maybe kinda makes sense
```

```
0 == False        # what?!
```

Ordering Predicates

The numeric types `int` and `float` have the following **ordering predicates**:

```
_<_      # is less than  
_>_      # is greater than
```

These predicates can also be used with strings following **lexicographic order**, a.k.a. “dictionary order”, *except* that *all* uppercase letters come before *any* lowercase letters.

```
"aardvark" < "zebra"  
"bow" < "bowdoin"  
"Zebra" < "aardvark"
```

Activity: Case-Insensitive String Comparison

Write a function `before : (str , str) --> bool` that returns whether the first string comes before the second one according to dictionary order, but ignoring upper-and-lower case.

For example:

```
before("aardvark" , "zebra")
```

```
True
```

```
before("Zebra" , "aardvark")
```

```
False
```

Boolean Operators

We have met some **numeric operators** like `_+_`, `_-_`, and `_*_` that let us build compound *numeric expressions*.

Python provides the following **boolean operators** to combine *boolean expressions* in a similar way: `_and_`, `_or_`, and `not_`.

Their values are determined according to the following **truth tables**:

↓ and →	True	False
True	True	False
False	False	False

↓ or →	True	False
True	True	True
False	True	False

b	not b
True	False
False	True

Compound Predicates

Using these, we can define functions that act like the built-in operators:

```
_<=_    # is less than or equal to  
_>=_    # is greater than or equal to  
_!=_     # is not equal to
```

For example:

```
def leq(num_1 , num_2) :  
    return num_1 < num_2 or num_1 == num_2  
  
def neq(num_1 , num_2) :  
    return not num_1 == num_2
```

Activity: Compound Predicates

Write a predicate function `is_even : (int) --> bool` that determines whether its argument is even.

(hint: the remainder operator `_%` will be helpful here.)

Write a predicate function `last_digit : (int) --> int` that gets the last digit of its (non-negative) argument number.

(hint: the remainder operator `_%` will again be helpful here.)

Use these two functions to write a predicate function

`is_even_but_doesnt_end_in_4 : (int) --> bool`
that returns `True` just in case the argument number is even
but its last digit is not 4.

Function Constraints

A function **constraint** is a predicate that the arguments must satisfy in order for the function to work as intended.

For example, we learned in elementary school that we can't divide by zero. If we ask python to do so, we will get an error.

We can make this restriction explicit using an **assertion** statement at the beginning of a function:

```
assert <predicate_on_arguments> , <error_message>
```

Safe Division

```
def remainder(number , denom) :  
    """ signature: (int , int) --> int """  
    assert denom != 0 , "can't divide by 0"  
    return number % denom
```

If the constraint is satisfied, the function behaves normally.

But violating the constraint results in an error letting us know that the function is not being used as intended.

Activity: Last String Character

Write a function `last_char : (str) --> str` that returns the last character of the argument string.

for example:

```
last_char("Bowdoin")
```

```
'n'
```

```
last_char("Hi!")
```

```
'!'
```

What constraint do we need on the argument in order for this to be possible?

Lazy Evaluation

Recall the boolean operators `_and_` and `_or_`:

$\downarrow \text{ and } \rightarrow$	True	False
True	True	False
False	False	False

$\downarrow \text{ or } \rightarrow$	True	False
True	True	True
False	True	False

For efficiency, python evaluates their arguments from-left-to-right, but stops as soon as it has enough information to know the result.

This makes a difference *only* if evaluating an argument causes some *effects*.

```
def p_n_r(x) : # print and return a value
    print("printing " + str(x))
    return x
```

Compare:

```
p_n_r(True) and p_n_r(False)
p_n_r(True) or p_n_r(False)
```

To Do

Finish *Homework 2: Strings* on CodeRunner by 11:00AM this Thursday.